



CHANNEL Z0 – STATION BUILD GUIDE

A local channel, for locals. Broadcasting from a house, relayed through a small server, watchable by everyone.

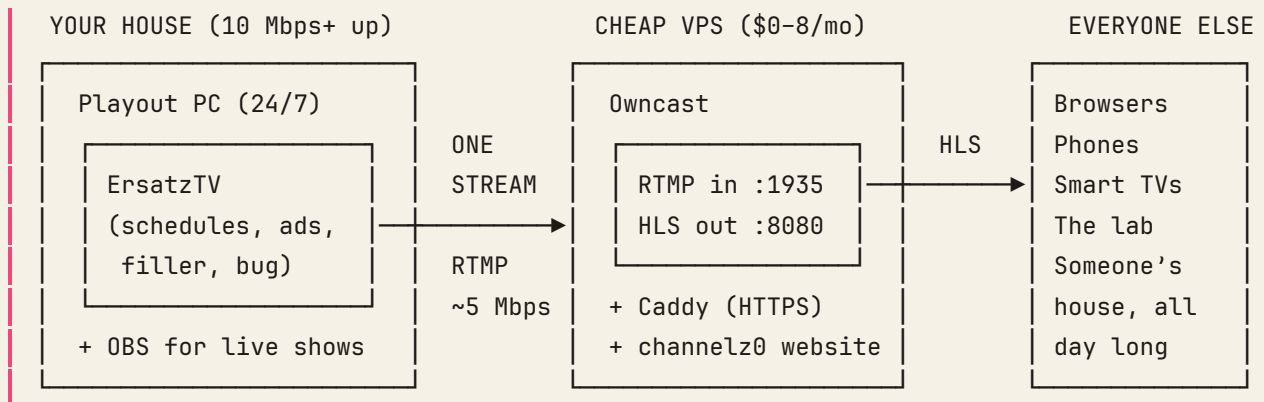
CONTENTS

▶ What you're building	3
▶ Phase 0 – Names and accounts (30 minutes)	4
▶ Phase 1 – The tower (VPS: Owncast + Caddy)	4
1.1 Install Docker and Caddy	5
1.2 Owncast	5
1.3 Caddy (HTTPS for free, automatically)	5
1.4 Lock the doors	6
1.5 Prove the pipe works	7
▶ Phase 2 – Master control (playout PC: ErsatzTV)	8
2.1 Lay out the media library	8
2.2 Install ErsatzTV (Docker)	9
2.3 Configure the station	10
▶ Phase 3 – The uplink (one relay, running forever)	12
▶ Phase 4 – The storefront (channelz0.example)	14
When channelz0.tv goes live	15
▶ Phase 5 – Going live (Ground Zero remotes, Lab Hour)	16
▶ The ad pipeline (locals → airwaves)	17
▶ Broadcast polish	17
▶ Reliability & operations	19
▶ Troubleshooting	20
▶ Command cheat sheet	22

This guide lives inside the Channel Z0 repository. Every config it builds by hand also ships as a ready-made file – look for the ♦ **in this repo** notes – and the station scripts referenced throughout are in `tools/`.

SEC.01

WHAT YOU'RE BUILDING



The entire trick of the station is in the middle column: **exactly one stream ever leaves your house**. The VPS does the crowd work. Your upload never sees viewer number two.

Why not WebRTC: it's built for sub-second, two-way calls, and every viewer would cost you upload bandwidth. A linear channel doesn't care about 20 seconds of delay – HLS trades latency you don't need for compatibility and scale you do. Old-style TV, boring modern plumbing.

The software, all free and open source:

PIECE	RUNS WHERE	JOB
ErsatzTV	Playout PC at home	Turns folders of video into a scheduled linear channel with ad breaks and a channel bug
FFmpeg	Playout PC at home	Relays the channel to the VPS, and preps submitted ads
Owncast	VPS	Accepts your one RTMP stream, serves HLS to everyone, ready-made watch page + chat
Caddy	VPS	HTTPS, hosts the Channel Z0 website
OBS Studio	Home / the lab	Live segments – Ground Zero remotes, lab shows

(Alternative playout: Tunarr does the same job as ErsatzTV; pick whichever UI you vibe with. This guide assumes ErsatzTV.)

Build order matters: **tower first, then master control**. Get the VPS receiving and re-serving video on day one with a junk test stream; build the actual channel second. You'll always have a working end-to-end path to test against.

SEC.02 PHASE 0 – NAMES AND ACCOUNTS (30 MINUTES)

1. **Domain.** `channelz0.tv` is the dream (~\$30-40/yr); `channelz0.org` or `.net` is ~\$12-15/yr and just as functional. Any registrar (Cloudflare, Porkbun, Namecheap).
2. **VPS.** See the companion **Gear & Cost Plan** for the comparison. Short version: Oracle Cloud's Always Free ARM tier (\$0, 10 TB/mo egress) or a ~\$6/mo box with a big traffic allowance. 1-2 vCPU and 2 GB RAM is plenty because the VPS never transcodes – it just repackages your already-encoded stream.
3. **DNS.** Point two records at the VPS IP:
 - `channelz0.example` → A record → VPS IP (the website)
 - `watch.channelz0.example` → A record → VPS IP (Owncast)

(Everywhere below, replace `channelz0.example` with your real domain.)

SEC.03 PHASE 1 – THE TOWER (VPS: OWNCAST + CADDY)

SSH into a fresh Ubuntu/Debian VPS.

1.1 INSTALL DOCKER AND CADDY

BASH

```
curl -fsSL https://get.docker.com | sh
sudo apt install -y caddy
```

1.2 OWNCAST

BASH

```
mkdir -p /opt/owncast && cd /opt/owncast
cat > docker-compose.yml <<'EOF'
services:
  owncast:
    image: owncast/owncast:latest
    container_name: owncast
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:8080"    # web + HLS, kept behind Caddy
      - "1935:1935"            # RTMP ingest, open to the world (key-protected)
    volumes:
      - ./data:/app/data
EOF
docker compose up -d
```

◆ **In this repo:** `vps/docker-compose.yml` – `scp` it up instead of typing.

1.3 CADDY (HTTPS FOR FREE, AUTOMATICALLY)

BASH

```
cat > /etc/caddy/Caddyfile <<'EOF'
channelz0.example {
  root * /srv/channelz0
  file_server
}
```

```
watch.channelz0.example {
    reverse_proxy 127.0.0.1:8080
}
EOF
mkdir -p /srv/channelz0
sudo systemctl reload caddy
```

◆ **In this repo:** `vps/Caddyfile`.

Caddy fetches TLS certificates on its own. `https://watch.channelz0.example` now shows Owncast's page (offline, for the moment). Drop the `channel-z0-site.html` file in as `/srv/channelz0/index.html` when you're ready – that's the storefront (Phase 4).

1.4 LOCK THE DOORS

Rather than clicking through the admin UI, do all three at once – and note the third, which the checklist never used to mention and which matters more than the other two:

BASH

```
Z0_OWNCAST_ADMIN_PASS=<current> tools/z0-tower-config.sh \
  --watch-domain watch.channelz0.example \
  --admin-pass <new> --stream-key <long-random>

# and afterwards, any time you suspect drift:
tools/z0-tower-config.sh --watch-domain watch.channelz0.example --check
```

Video passthrough must be on. It is the whole reason a 1 GB free-tier box can carry a television channel. With it off, Owncast software-encodes the incoming stream; on a micro shape that runs at about 0.93x realtime, so the public stream falls behind roughly six seconds per minute – forever, because the uplink is capped at 1.0x and every second lost is lost. It does not present as an encoder fault. It presents as "the stream is a couple of minutes behind", then ten, while `/api/status` says `online:true` throughout.

Changing it does **not** restart the running transcode: Owncast only builds a new pipeline when a new RTMP session starts, so the config reads back correct while

the old ffmpeg keeps going. Restart the uplink on the playout node (`docker restart z0-uplink`) to make it take.

Open `https://watch.channelz0.example/admin` (default login is `admin / abc123` on current builds – the console log tells you if it differs) and immediately:

1. **Change the admin password and the stream key.** The stream key is the only thing standing between the world and your airwaves. Make it long and random.
2. Set the server name (**Channel Z0**), logo, and tags.
3. Under video settings, set the output to a **single quality with video passthrough** – Owncast then repackages your incoming stream without re-encoding it, and the cheapest VPS never breaks a sweat. (Only add a transcoded lower rung later if phone viewers on bad connections complain – that's when you'd want 2+ vC-PUs.)
4. Set the **offline message** – this is what plays between transmissions: **"TRANSMISSION INTERRUPTED – Z0 RETURNS SHORTLY."**
5. Firewall, if the host has one: allow 22, 80, 443, 1935.

1.5 PROVE THE PIPE WORKS

From any machine with FFmpeg, send colour bars at the tower:

BASH

```
ffmpeg -re \  
  -f lavfi -i "smptehdbars=size=1280x720:rate=30" \  
  -f lavfi -i "sine=frequency=440:sample_rate=48000,volume=0.1" \  
  -c:v libx264 -preset veryfast -b:v 2500k -pix_fmt yuv420p \  
  -c:a aac -b:a 96k \  
  -f flv rtmp://watch.channelz0.example:1935/live/YOUR_STREAM_KEY
```

◆ **In this repo:** `tools/test-broadcast.sh` runs this for you, reading your domain and key from `.env`.

If `watch.channelz0.example` shows bars and hums quietly at you within ~30 seconds, the tower is live. Everything after this is programming.

SEC.04 PHASE 2 – MASTER CONTROL (PLAYOUT PC: ERSATZTV)

The playout PC is the machine that runs 24/7 at home. Any x86 box with Intel Quick Sync (or an NVIDIA card) works; see the **Gear & Cost Plan**. Wire it to the router with ethernet. Wi-Fi is where 24/7 streams go to die.

2.1 LAY OUT THE MEDIA LIBRARY

Structure the folders the way the station thinks:

```

/media/channelz0/
├── shows/
│   ├── Ground Zero/
│   │   └── Season 01/
│   │       ├── Ground Zero - s01e01 - First Contact, First Baguette.mp4
│   │       └── Ground Zero - s01e02 - The Skin Fits Fine.mp4
│   └── Lab Hour/
│       └── Season 01/...
├── movies/                                # public-domain features
├── commercials/
│   ├── local/                            # submitted spots (screened, normalized)
│   └── lab/                               # your sponsor promos
├── bumpers/                              # "You're watching Channel Z0" (5-15s each)
├── psas/                                  # Prelinger-archive vintage PSAs & shorts
├── interstitials/
│   ├── colorbars-1h.mp4                  # sign-off loop (generated below)
│   └── technical-difficulties.mp4

```

Shows use Plex-style naming (Show - s01e01 - Title.ext) so the library scanner files them correctly.

◆ **In this repo:** `tools/make-media-tree.sh` creates this whole layout in one go.

Free period-correct filler: the Internet Archive's Prelinger collection – thousands of public-domain educational films, PSAs, and vintage ads. Exactly the texture an old-style channel needs between local spots, and legally spotless.

◆ **In this repo:** `tools/fetch-archive.sh` does the fetching – Prelinger shorts, cartoons, and the themed feature nights, filtered to public-domain-marked items and verified with `ffprobe` on the way in. It is resumable, so run it overnight against the colour bars: `tools/fetch-archive.sh --all`. Full manual in `docs/archive-fetch.md` `RL-Z0-ARCHIVE-FETCH`.

▲ Don't point a fetcher at `archive.org`'s `classic_tv_commercials` collection wholesale – it mixes genuine vintage spots with material still very much in copyright (there's a 2007 FedEx ad in there). Filter on the licence mark, which is what the tool does.

2.2 INSTALL ERSATZTV (DOCKER)

BASH

```
docker run -d --name ersatztv \
  -p 8409:8409 \
  -v /opt/ersatztv/config:/config \
  -v /media/channelz0:/media:ro \
  --device /dev/dri:/dev/dri \
  --restart unless-stopped \
  ghcr.io/ersatztv/ersatztv:latest-vaapi
```

◆ **In this repo:** `playout/ersatztv.sh` wraps this, with the tag configurable via `ERSATZTV_TAG`.

Notes, as of this writing: `latest-vaapi` is the right tag for Intel Quick Sync boxes (`latest-nvidia` for NVENC; plain `latest` for software-only). `--device /dev/dri` hands the GPU to the container. Web UI lands on `http://playout-pc:8409`.

2.3 CONFIGURE THE STATION

If a Channel Z0 already exists somewhere, do not do any of this by hand. Capture that station and pour it into this box – it carries the profile, the libraries, the collections, the filler presets, the channel, the watermark, the graphics elements and the playout, and it re-points the encoder at whatever hardware this machine has:

BASH

```
# on the machine that is on air – read-only, safe against a live station
tools/z0-station-image.sh --capture --from root@<node>:/config --out /tmp/z0-station.tgz
# on the new box
tools/bootstrap-node.sh --role playout --station-image /tmp/z0-station.tgz ...
```

See clustering_{RL-Z0-CLUSTERING}. There is no config API to script instead – ErsatzTV's `/api/...` serves the SPA, not JSON – so the database is the station.

The steps below are how you build the **first** one, from nothing.

In the ErsatzTV web UI, in order:

1. **FFmpeg profile** (Settings → FFmpeg Profiles): one profile, Z0 Broadcast – 1920×1080, H.264 hardware encode (VAAPI/QSV), ~4500 kbps video, AAC 128 kbps audio, normalize framerate on. If your uplink is a true 10 megabits, make it 1280×720 at 3000–3500 kbps instead and leave headroom for the household. This one profile is the master encode – everything downstream just copies it.
2. **Libraries** (Media → Libraries): add local libraries pointing at `shows/`, `movies/`, and – as **Other Videos** – `commercials/`, `bumpers/`, `psas/`, `interstitials/`. Set libraries to rescan on an interval, so dropping a new ad in the folder is all it takes.
3. **Collections**: build collections named `Local Ads`, `Lab Promos`, `Bumpers`, `Vintage PSAs`, `Colour Bars`. These are the ammunition for filler.
4. **Filler presets** (Media → Filler Presets): this is the old-TV magic.
 - `Ad Break` – mid-roll filler drawing from `Local Ads` + `Lab Promos` + `Vintage PSAs`.
 - `Station ID` – pre/post-roll from `Bumpers`.

-
- ▶ **Pad to the half hour:** use padded filler mode so every slot fills to :00/:30 with ads. This is the single feature that makes it feel like real television – shows start on the half hour **because the ad break stretched to fit**, exactly like 1994.
 - ▶ **Dead Air** – fallback filler set to **Colour Bars**, so if a schedule ever runs dry the station shows bars instead of black.
5. **Channel** (Channels → Add): number **1**, name **Channel Z0**, streaming mode **MPEG-TS**, your **Z0 Broadcast** profile. Add the **watermark**: your Z0 logo PNG, bottom-right, ~15% opacity, always on. That's the channel bug.
 6. **Schedule** (Schedules): build the broadcast day from schedule items (each item = a collection or show + a filler preset). A starting rhythm:
 - ▶ mornings: cartoons/PSAs · midday: Prelinger theatre + community loop · afternoon: movie
 - ▶ **19:00 – GROUND ZERO** (flagship slot, protect it)
 - ▶ 20:00: prime movie · 23:00: mellow late block
 - ▶ ♦ **In this repo:** `docs/programming.md`^{RL-Z0-PROGRAMMING} turns that rhythm into the whole week – every block defined, the themed nights, the show bible, and the exact ErsatzTV mapping (collections, fixed-start anchors, weekend schedule). It's the twin of the storefront's program grid; build from it.
 - ▶ **00:00 – SIGN-OFF:** a nightly ritual item (anthem, station sign-off card – `tools/make-signoff.sh` generates the card-into-bars close), then **Colour Bars** until the 06:00 sign-on. Deeply on-brand, and it costs nothing.
 7. Sanity-check locally: open `http://playout-pc:8409/iptv/channel/<number>.ts` in VLC – **the channel number you set in step 5, not 1**. Channel Z0 is number 0, and `/iptv/channel/1.ts` returns 404 on it; a wrong number here gives you a node that answers every health check and publishes nothing. You should be watching Channel Z0. (ErsatzTV also serves an XMLTV guide at `/iptv/xmltv.xml` – useful later for generating the website's schedule.)

SEC.05 PHASE 3 – THE UPLINK (ONE RELAY, RUNNING FOREVER)

The relay is a single FFmpeg process that copies the already-encoded channel from ErsatzTV to the tower. `-c copy` means no second encode – it's pure plumbing, nearly zero CPU.

The unit ships in this repo as `playout/z0-uplink.service`. It reads your domain, stream key, and channel URL from an environment file, so the secret never lives inside the unit:

INI

```
[Unit]
Description=Channel Z0 uplink (ErsatzTV -> Owncast)
After=network-online.target docker.service
Wants=network-online.target

[Service]
EnvironmentFile=/etc/channel-z0.env
ExecStart=/usr/bin/ffmpeg -hide_banner -loglevel warning \
  -i ${Z0_CHANNEL_URL} \
  -c copy \
  -f flv rtmp://${Z0_WATCH_DOMAIN}:1935/live/${Z0_STREAM_KEY}
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Install it:

BASH

```
sudo cp playout/z0-uplink.service /etc/systemd/system/
sudo cp .env.example /etc/channel-z0.env # then edit in your real values
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now z0-uplink
```

Restart=always is the station engineer who never sleeps: home internet blips, ErsatzTV restarts, whatever – the uplink reconnects five seconds after the path returns, and Owncast picks the stream back up. While it's down, viewers see your offline message instead of an error. Broadcast continuity, handled.

Uplink budget check: 4500 kbps video + 128 kbps audio + overhead $\approx$ 5 Mbps sustained. On an 80 Mbps (10 MB/s) uplink that's 6% utilization; even on a literal 10 Mbps plan it fits with room for the household, at 720p. One stream. Always one stream.

Containerized alternative. Instead of the systemd unit, `playout/compose.yml` runs the whole home side – ErsatzTV, the uplink relay, and the optional now-playing bridge – in one `docker compose up -d`. It's the easy path on a homelab; see `docs/self-hosting.md` RL-Z0-SELF-HOSTING for Proxmox and TrueNAS specifics (mostly: how to hand the box's iGPU to the container).

The live marquee (optional). Owncast doesn't know what ErsatzTV is airing – `playout/nowplaying.py` closes that gap. It reads ErsatzTV's XMLTV guide, works out the current program, and writes it into Owncast's stream title, which the storefront then shows as a live NOW SHOWING line. Create an Owncast access token at `https://watch.<domain>/admin/access-tokens` with the **set stream title** scope, put it in `.env` as `Z0_OWNCAST_TOKEN`, and start it with `docker compose --profile nowplaying up -d` (or run the script under a systemd timer).

SEC.06 PHASE 4 – THE STOREFRONT (CHANNELZ0.EXAMPLE)

The storefront lives at `site/index.html` – live player, ON AIR light, program grid, and the ad-submission counter, in the Riposte Laboratories design language. (The original CRT version is preserved at `site/retro/index.html`.) It's hosted on **Cloudflare** (a Worker serving `site/` as static assets) – the tower (Owncast) carries only the video, and Cloudflare carries the page. To deploy:

1. Open the file and edit the two lines at the top of the `<script>` – everything in `CONFIG` derives from them: `js const WATCH_HOST = "watch.ch0.ripostelabs.xyz"; // the Owncast tower on the VPS const AD_EMAIL = "ch0@ripostelabs.xyz"; // where spot submissions land`
2. **Deploy to Cloudflare** (Workers Static Assets via its Git integration – no CI file, no secrets):
 - ▶ Workers & Pages → **Create** → **Import a repository** → pick your repo. Cloudflare detects the static site, commits a `wrangler.jsonc` (with `assets.directory: "site"`), and deploys. You get a `*.workers.dev` URL.
 - ▶ The Worker → **Settings** → **Domains & Routes** → add a **Custom Domain** `ch0.ripostelabs.xyz` (instant, since `ripostelabs.xyz` is on Cloudflare DNS).
 - ▶ Every push to `main` now auto-deploys. (Prefer deploying by hand? `npx wrangler deploy`.) **DNS gotcha for the tower:** add `watch.ch0.ripostelabs.xyz` → VPS IP as **DNS only (grey cloud)**, **not proxied** – Cloudflare's proxy won't pass RTMP (`:1935`) and shouldn't carry a 24/7 video stream. The page is proxied; the stream host is not.
3. **Cross-origin note.** The page is on Cloudflare and the stream is on the VPS, so the tower must send permissive CORS – the updated `vps/Caddyfile` does this. `site/_headers` and `site/_redirects` (security headers, and short links like `/watch`) are honoured by Workers Static Assets, same as Pages. There is deliberately **no** GitHub Pages fallback: GitHub Pages is not enabled on this repo, and Cloudflare is the only publishing route.

WHEN CHANNELZ0.TV GOES LIVE

Everything is staged for a one-line flip; do these in order:

1. **Buy it** at any registrar, then add `channelz0.tv` to Cloudflare (Add a site) so its DNS is Cloudflare-managed.
2. **Custom domains.** In the `channel-z0` Worker → Settings → Domains & Routes, add `channelz0.tv` (and `www` if you want) as Custom Domains. Both it and `ch0.ripostelabs.xyz` now serve the site.
3. **The tower.** Add a DNS record `watch.channelz0.tv` → VPS IP, and uncomment the `watch.channelz0.tv { ... }` block in `vps/Caddyfile`; `sudo systemctl reload caddy`. Caddy fetches the cert automatically.
4. **The site.** In `site/index.html`, change the two lines at the top of the script – `WATCH_HOST = "watch.channelz0.tv"` and `AD_EMAIL = "ads@channelz0.tv"`. Everything else in `CONFIG` derives from those. Push; Cloudflare redeploys on its own.
5. **.env** on the playout box: set `Z0_SITE_DOMAIN` / `Z0_WATCH_DOMAIN` to the `channelz0.tv` hostnames (and re-issue the Owncast token against the new host if you moved it).
6. **Optional – canonicalize.** To make `channelz0.tv` the one true home and redirect the old subdomain, uncomment the 301 in `site/_redirects`.

The player uses `hls.js` and falls back to native HLS on Safari/iPhones. If the video ever refuses to load cross-subdomain (a CORS grump), the two-line fallback is an `iframe` of Owncast's built-in player: `<iframe src="https://watch.channelz0.example/embed/video" allowfullscreen></iframe>` – but the direct HLS route is the full retro experience, so try that first. Owncast's own page at `watch.` stays useful regardless: it has the live chat.

SEC.07 PHASE 5 – GOING LIVE (GROUND ZERO REMOTES, LAB HOUR)

Two patterns, in order of ambition:

Pattern A – the hard cutover (start here). OBS on any machine, streaming to the **same** RTMP URL and key. When it's time to go live:

BASH

```
sudo systemctl stop z0-uplink      # network feed off
# OBS: Start Streaming              live from the lab / the field
# ...show happens...
# OBS: Stop Streaming
sudo systemctl start z0-uplink     # network feed returns
```

Viewers see a few seconds of "transmission interrupted" at each swap – which, for a channel run by an alien in a human suit, is arguably canon.

Pattern B – OBS as master control (the upgrade). OBS runs 24/7 on the playout PC and **it** owns the uplink; the ErsatzTV feed becomes just another input:

- ▶ Scene 1 **NETWORK FEED**: a VLC/media source playing `http://127.0.0.1:8409/iptv/channel/1.ts`
- ▶ Scene 2 **LIVE – GROUND ZERO**: camera + the baguette feed
- ▶ Scene 3 **TECHNICAL DIFFICULTIES**: a slate card, for emergencies

Now going live is one seamless scene-click, no dropout. Cost: OBS re-encodes the feed continuously, so the playout PC needs its hardware encoder for OBS (set OBS output to QSV/NVENC, 4500 kbps, and have ErsatzTV's profile use software or the other encoder). Do Pattern A until the swap dance annoys you; graduate to B.

Field kit notes for Ground Zero – including how to hide a lavalier microphone inside a real baguette – are in the **Gear & Cost Plan**.

SEC.08

THE AD PIPELINE (LOCALS → AIRWAVES)

1. **Intake.** Publish specs on the website (the site file has them built in): 15 / 30 / 60 seconds exactly, MP4 (H.264 + AAC), 1080p or 720p, you must own the rights to everything in it, keep it neighbourly. Collect via `ads@channelz0.example` or a free file-drop form (Google Form works fine).
2. **Screening.** Watch every submission before it airs. You are the FCC of this operation. The written one-page standard that saves arguments later ships as `docs/ad-standards.md` `RL-Z0-AD-STANDARDS` – post it, link it, point to it.
3. **Normalize.** Every accepted spot gets one pass through this – same resolution, same framerate, same **loudness** (the classic sin of TV ads was being louder than the show; you get to fix history): `bash ffmpeg -i submission.mp4 \ -vf "scale=1920:1080:force_original_aspect_ratio=decrease,pad=1920:1080:(ow-iw)/2:(oh-ih)/2,fps=30,format=yuv420p" \ -af "loudnorm=I=-16:TP=-1.5:LRA=11" \ -c:v libx264 -preset slow -b:v 5000k -maxrate 6000k -bufsize 10000k \ -c:a aac -b:a 128k -ar 48000 -movflags +faststart \ "commercials/local/BusinessName - Spot Title (30s).mp4" ♦ In this repo: tools/normalize-ad.sh runs this and files the result in the right folder: tools/normalize-ad.sh submission.mp4 "BusinessName - Spot Title (30s)".`
4. **Air.** Drop the file in `commercials/local/`. The library rescan picks it up; the Local Ads collection includes it; the Ad Break filler starts dealing it into rotation. No further ceremony.

SEC.09

BROADCAST POLISH

Generate the sign-off colour bars (one hour of SMPTE bars with a polite, quiet hum – swap the sine for `anullsrc=r=48000:cl=stereo` if you want silence):

BASH

```
ffmpeg \
  -f lavfi -i "smpthdbars=size=1920x1080:rate=30" \
```

```
-f lavfi -i "sine=frequency=440:sample_rate=48000,volume=0.05" \
-c:v libx264 -preset veryfast -b:v 2000k -pix_fmt yuv420p \
-c:a aac -b:a 96k -t 3600 \
interstitials/colorbars-1h.mp4
```

◆ **In this repo:** `tools/make-colorbars.sh` – pass minutes as an argument; `Z0_SILENT=1` swaps the hum for silence.

The test card. Raw colour bars prove the pipe (Phase 1.5), but the thing you show **before sign-on** is the card with the station's name on it – geometry grid, colour bars, a greyscale step wedge, corner castellations, a centre identity panel, and a 1 kHz line-up tone. `tools/make-testcard.sh` generates it at 1080p in the station's look; ErsatzTV loops the short clip to fill the pre-06:00 slot, and it doubles as a "please stand by" with a picture the tower can align to. `Z0_SILENT=1` drops the tone.

The sign-off. The end-of-broadcast-day close for the 00:00 SIGN-OFF item: the "THIS CONCLUDES OUR BROADCAST DAY / GOODNIGHT, LOCALS" card dissolving into a short tail of colour bars, so it hands off cleanly to the Colour Bars filler that carries the rest of the night. `tools/make-signoff.sh` makes the whole close in one file – `tools/make-signoff.sh 25 10` for a 25-second card and a 10-second bars tail. Where `make-slate.sh` makes a static sign-off **card**, this is the ritual, card into bars.

Bumpers. Five to ten seconds, "NOW WATCHING CHANNEL Z0," made in anything (Kdenlive/DaVinci Resolve are free) – or generated on brand in one command with `tools/make-ident.sh`. Record/generate a few variants; the Station ID filler rotates them. This is 80% of what makes a stream feel like a **station**.

Slates. The technical-difficulties card, the sign-off card, "please stand by" – `tools/make-slate.sh` generates any of them at 1080p in the station's look. Fills the `interstitials/technical-difficulties.mp4` the media tree expects.

The bug. The watermark you set in Phase 2 runs always. Semi-transparent, bottom-right, never explained. Non-negotiable old-TV physics. `tools/make-bug.sh` generates the transparent PNG to point the watermark at.

Screening spots. Before you accept a submission, `tools/check-ad.sh` reads its length, codecs, and true loudness so you know what you're clearing; `tools/normalize-ad.sh` then files it for air.

Loudness discipline. Run **everything** – shows, bumpers, ads – through the same `loudnorm` settings so channel-surfing ears never get blasted. Consistency is the retro luxury.

SEC.10

RELIABILITY & OPERATIONS

- ▶ **When home internet drops:** uplink dies → Owncast flips to your offline message → uplink service reconnects automatically when the line returns. Viewers refresh and they're back. No action required at 3am.
- ▶ **When the power blinks:** a small UPS (~\$60) on the modem + router + playout PC rides through the flickers. Everything is set to `--restart unless-stopped / Restart=always`, so even a full outage self-heals on power-up.
- ▶ **Monitoring:** Owncast's admin shows live viewer counts and stream health. Point free UptimeRobot at `https://watch.channelz0.example/api/status` – but make it a **keyword** monitor on `"online":true`, not a plain up/down check. A plain check watches the **tower**, and the tower is almost never the thing that breaks: Owncast, Caddy and the VPS stay perfectly healthy and keep answering 200 while the house has stopped sending anything at all. Ask it whether the station is **broadcasting**, not whether the server is **answering**. We learned this the expensive way – the channel sat dark for twenty hours behind a completely green endpoint.
- ▶ **Updates:** monthly-ish, `docker compose pull && docker compose up -d` on the VPS, `docker pull ghcr.io/ersatztv/ersatztv:latest-vaapi` + recreate at home. Don't update on a Friday before a Ground Zero premiere.
- ▶ **Legal footing:** submitted ads (with a rights attestation), your own shows, and public-domain material keep you clean. Copyrighted music and TV shows are the one real wrinkle – a "local channel for locals" is exactly the kind of thing that stays charming by staying legitimate.

SEC.11

TROUBLESHOOTING

SYMPTOM	LIKELY CAUSE → FIX
Viewers buffer every few seconds	Uplink saturated: drop the FFmpeg profile to 720p/3500k, check nothing else at home is uploading, use ethernet
Stream hiccups at every show change	Ensure the single FFmpeg profile + normalize framerate is on, so ErsatzTV's output stays codec-identical across items; the uplink's Restart=always covers the rest
Uplink log: "non-monotonic DTS" warnings	Cosmetic at this scale; if FFmpeg actually exits, add <code>-fflags +genpts</code> before <code>-i</code>
Owncast rejects reconnect right after a drop	It holds the old session a few seconds; the service retries every 5s and wins – wait one cycle
Website player black, but Owncast page works	STREAM_URL typo or CORS: test the m3u8 in VLC, or fall back to the iframe embed
Ads much louder/quieter than shows	A spot skipped the <code>loudnorm</code> normalize pass – re-run it
"It's 25 seconds behind real time!"	Yes. It's television. Latency is the cost of infinite viewers on one home connection
Tower says "online": false, but the VPS and Owncast are fine	Nothing is arriving. Check the uplink's channel URL: <code>/iptv/channel/N.ts</code> takes the channel's number , not its id, so renumbering a channel in ErsatzTV silently breaks the relay – the URL starts returning 400, the leader sees an unhealthy channel and stands down. Confirm with <code>curl -o /dev/null -w '%{http_code}' http://127.0.0.1:8409/iptv/channel/N.ts</code> and compare against <code>/iptv/channels.m3u</code>
A block is missing from the guide entirely	Its content pool resolved to zero items, which is silent – the fill loop is guarded on the enumerator having a current item, so an empty pool schedules nothing and leaves a hole. Either the folder is empty, or the library was scanned but the search index was not (see Phase 2)
A block is short and hands off to filler early	<code>pad_until</code> only schedules an item that FITS the remaining time, and gives up on the first one that doesn't. Add <code>discard_attempts</code> so it looks further down the pool – essential when the pool's runtimes are longer than the slot

SEC.12

COMMAND CHEAT SHEET

DO	WHERE	COMMAND
Watch the channel raw	anywhere local	<code>vlc http://playout-pc:8409/iptv/channel/1.ts</code>
Uplink status / logs	playout PC	<code>systemctl status z0-uplink · journalctl -fu z0-uplink</code>
Stop/start network feed	playout PC	<code>sudo systemctl stop z0-uplink / start</code>
Owncast logs	VPS	<code>docker logs -f owncast</code>
Normalize a submitted ad	anywhere	see The ad pipeline §3
Regenerate sign-off bars	anywhere	see Broadcast polish
Generate the test card	anywhere	<code>tools/make-testcard.sh</code>
Generate the nightly sign-off	anywhere	<code>tools/make-signoff.sh</code>
Test pattern to the tower	anywhere	see Phase 1.5

Channel Z0. It's always on somewhere in the neighbourhood.

SOURCE docs/build-guide.tex SHA 8efe3ab68ee5 BUILT 2026-09-07